HPC on-demand: Dynamic on-demand cluster - computing for SMEs on COTS computers

Introduction

⚠ Data Science is rarely applied in small to medium-sized enterprises due to the lack of dedicated server-administration and/or hardware.

💡 Reducing the investment cost of infrastructure could motivate the exploration of data science and machine learning solutions that may improve existing processes and work, e.g. in quality control.



Approach

Build a reproducible and scalable Plug & Play Live-ISO with minimal configuration requirements that runs a multi purpose cluster on existing infrastructure in the SMEs, serving well established tools and frameworks that enable users to develop and train machine learning models. Thus, SMEs can use e.g. the developers' machines during off-business hours to train machine learning models, without the need for dedicated hardware or cloud solutions. This reduces overall costs and contributes to data security and protection of intellectual property.

Requirements

> Boot Media, i.e. USB Sticks
> Minimum consumer grade pc, up to dedicated server hardware
> Tailscale account

What the Live-ISOS do

Headnode

> Directory setup for User homes, shared via NFS
> Provide job scheduling and resource allocation via Slurm Controller Node
> Provide JupyterHub on port 8000
> Launching Jupyterlabs via Slurm with preconfigured ready-to-go development environments.

Workernode

> Mount NFS share
> Run Slurm Compute Node and register to Slurm Controller Node

From zero to cluster

On your machine:

1. Install NIX package manager
2. Adjust the config.json to your needs (example shown below)
3. Run `nix build` and `nix build .#worker`

Via CI/CD pipeline:

1. Fork the repository
2. Adjust the config.json to your needs (example shown below)
3. Commit changes to trigger build-pipeline
4. Download the ISOs from your git platform

Burn the ISOs to USB sticks, plug them into a computer and boot them. The cluster will then be available.

```
[tailscale]
domain = "TAILSCALE_DOMAIN"
authKey = "TS_AUTHKEY"
apiKey = "TS_APIKEY"

[users.alice]
passwordHash = "$6$niENrHi8EzNkxcQb"
uid = 5001

[users.bob]
passwordHash = "$6$niENrHi8EzNkxcQb"
uid = 5002

[telemetry]
enabled = true
```

Example configuration file

Project links

https://fhac.de/FB05/hpc-on-demand

https://fhac.de/FB05/bigdata

https://fhac.de/FB05/maskor

Feature: Interactive programming

> Enable ad-hoc exploratory processes in Data Science
> Insights lead to iterations with new questions
> Data Scientists require interactive development environments

💡 JupyterLab is widely used for interactive development

Feature: Distributed Computing

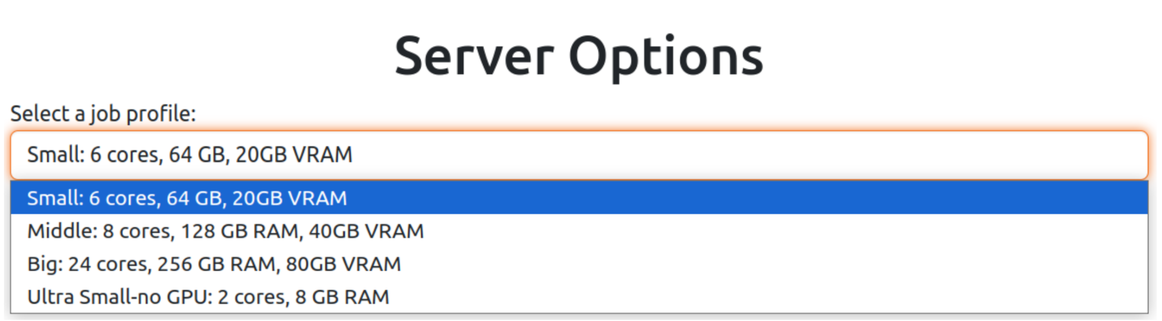
> Users might want to use as many resources as the cluster can offer them for one computation, e.g. training of a neural network using Distributed Data Parallel or preparing datasets

💡 Dask enables users to distribute their workloads over the whole cluster. Users can spawn dask clusters via slurm with customizable resource specifications. The Dask workers receive and execute workloads and send their results back to the user.

Feature: Specifying resources per session

> Companies face many different use cases with very different compute requirements
> Varying resource requirements between topics, project phases and sessions
> Using many resources blocks them for others

💡 Dynamic resource allocation can improve efficiency



Selection of different resource profiles for JupyterLab

Software Stack

NixOS

> Linux distribution with focus on reproducibility
> allows declaritive configurations and stable package management

Tailscale

> fully automated networking through tailscale and their magicDNS

Slurm

> fault-tolerant, highly scalable cluster management tool
> used as resource scheduler

Python & JupyterLab

> Interactive programming environment
> enables users to interactively develop on the cluster
> Users can manage their own python environment (e.g. install pytorch or tensorflow)

Dask

> flexible open-source python library for parallel computing
> allows distributing workloads onto multiple workernodes

Future Work

> Provide robust GPU support on worker nodes
> Utilize distributed filesystems to achieve data locality on worker nodes
> Streamline the configuration and build process with optional features

Funding

We acknowledge the support by the Federal Ministry of Education and Research (BMBF) under grant number 02L19C602 (project WIRKsam)

Authors

Max Conzen
Ingo Elsen
Oskar Galla
Marcel Remmy

Big Data Lab

MASKOR
Mobile Autonome Systeme und Kognitive Robotik
FH AACHEN UNIVERSITY OF APPLIED SCIENCES